

Lab Eleven – Point Estimation and Resampling

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with “**Solution:**”.
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use “enter” to make your code readable, so it doesn’t run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you’re submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don’t forget that you can copy-and-paste code when you’re doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good **ggplot** practice AND will help make sure you don’t make a mistake when computing the probability.

1 Introduction

When performing point estimation, there isn’t just one *right* answer. For example, the Method of Moments (MOM) and Maximum Likelihood Estimates (MLEs) aren’t always the same.

One question that comes up, is why we calculate the sample variance as

$$S_{n-1}^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2,$$

when it is supposed to be the “average squared distance from the mean, i.e.,

$$S_n^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2.$$

2 Comparing Estimators

2.1 Initial Simulation

In this lab, we will explore why we use S_{n-1}^2 and not S_n^2 . To do this, we will generate data from four known distributions:

- Poisson($\lambda = 2$)
- Binomial($n = 50, p = \frac{5-\sqrt{21}}{10}$)
- Normal($\mu = 0, \sigma = \sqrt{2}$)
- Exponential($\lambda = \frac{1}{\sqrt{2}}$).

Remark: Note that I have selected the parameters so that the true variance for all four models is 2.

For each distribution, write a loop that completes the following 1000 times. Put `set.seed(7272)` at the top of your code chunk.

- Generate a sample of size n from the distribution.
- Calculate and save S_{n-1}^2 and S_n^2 .

Start with $n = 20$. Is there evidence that S_{n-1}^2 is a better estimator than S_n^2 ?

Create a plot or combination of plots (using patchwork) to show the results across all four distributions. Make sure to create a corresponding table that numerically summarizes the results of the simulation. You should include

$$\text{Bias} = \text{Average of Estimates} - \text{True Value}$$

$$\text{Precision} = \frac{1}{\text{Variance of Estimates}}$$

$$\text{Mean Square Error} = \text{Bias}^2 + \text{Variance of Estimates}.$$

Solution

```
1 set.seed(7272)
2
3 library(ggplot2)
4 library(dplyr)
5 library(patchwork)
6
7 #Simulation Function
8 simulate.variance = function(generator.fn, n, n.sim = 1000) {
9   s_n1 = numeric(n.sim)
10  s_n = numeric(n.sim)
11
12  for (i in 1:n.sim) {
13    x = generator.fn(n)
14
15    s_n1[i] = var(x) # S_{n-1}^2
16    s_n[i] = mean((x - mean(x))^2) # S_n^2
17  }
18
19  tibble(s_n1 = s_n1, s_n = s_n)
20 }
```

```
1 #Generate data for each distribution
2 gen.pois = function(n) rpois(n, 2)
3 gen.binom = function(n) rbinom(n, 50, (5 - sqrt(21))/10)
4 gen.norm = function(n) rnorm(n, 0, sqrt(2))
5 gen.exp = function(n) rexp(n, rate = 1/sqrt(2))
6
7 #Simulations when n = 20
8 n = 20
9
10 pois.data = simulate.variance(gen.pois, n)
11 binom.data = simulate.variance(gen.binom, n)
12 norm.data = simulate.variance(gen.norm, n)
13 exp.data = simulate.variance(gen.exp, n)
```

```
1 #Summary Function
2 summarize.results = function(data, dist.name) {
3   true.var = 2
4
5   tibble(
6     Distribution = dist.name,
7     Estimator = c("S_{n-1}^2", "S_n^2"),
8     Bias = c(mean(data$s_n1) - true.var,
9              mean(data$s_n) - true.var),
10    Variance = c(var(data$s_n1),
11                 var(data$s_n)),
12    Precision = c(1 / var(data$s_n1),
13                  1 / var(data$s_n)),
```

```

14     MSE = c(
15       (mean(data$s_n1) - true.var)^2 + var(data$s_n1),
16       (mean(data$s_n) - true.var)^2 + var(data$s_n)
17     )
18   )
19 }

```

```

1 results.table = bind_rows(
2   summarize.results(pois.data, "Poisson"),
3   summarize.results(binom.data, "Binomial"),
4   summarize.results(norm.data, "Normal"),
5   summarize.results(exp.data, "Exponential")
6 )
7 results.table |>
8   knitr::kable(digits = 2)

```

Table 1: Comparison of Variance Estimators Using Bias, Variance, Precision, and MSE

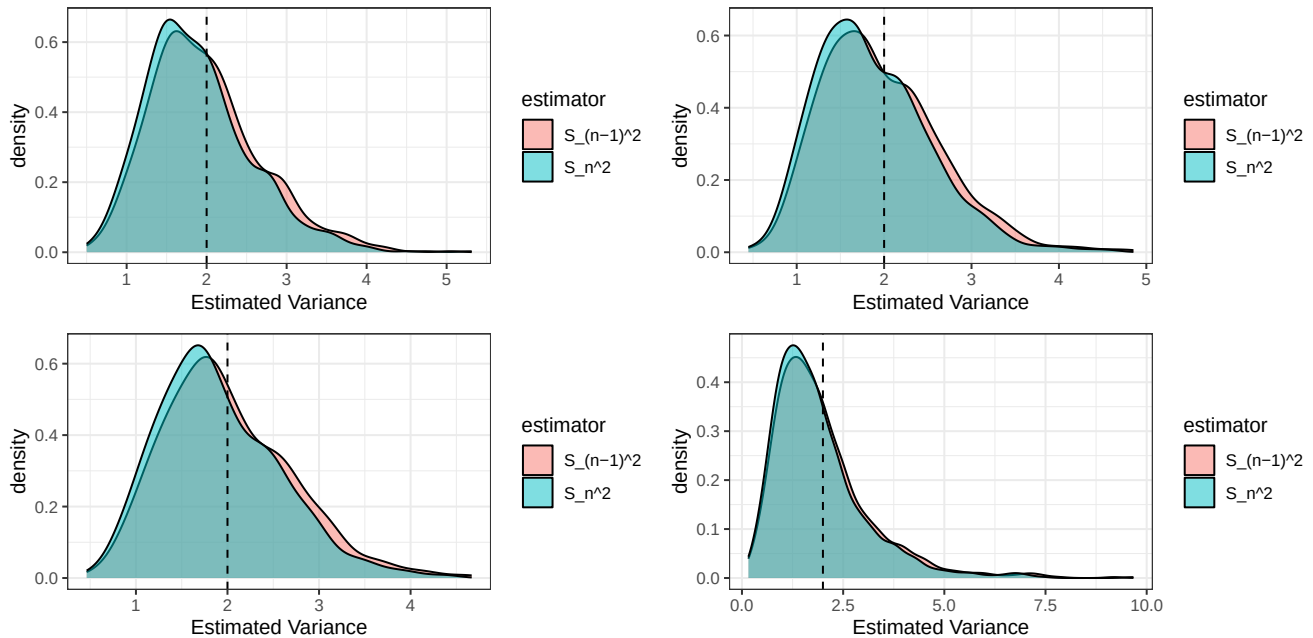
Distribution	Estimator	Bias	Variance	Precision	MSE
Poisson	S_{n-1}^2	-0.02	0.48	2.10	0.48
Poisson	S_n^2	-0.12	0.43	2.33	0.44
Binomial	S_{n-1}^2	-0.04	0.47	2.15	0.47
Binomial	S_n^2	-0.13	0.42	2.38	0.44
Normal	S_{n-1}^2	0.00	0.49	2.03	0.49
Normal	S_n^2	-0.10	0.44	2.25	0.45
Exponential	S_{n-1}^2	-0.02	1.43	0.70	1.43
Exponential	S_n^2	-0.12	1.29	0.77	1.30

```

1 #Plot Function
2 make.plot = function(sim.data, title) {
3   df.plot = tibble(
4     value = c(sim.data$s_n1, sim.data$s_n),
5     estimator = rep(c("S_(n-1)^2", "S_n^2"), each = nrow(sim.data))
6   )
7   #Plot
8   ggplot(df.plot, aes(x = value, fill = estimator)) +
9     geom_density(alpha = 0.5) +
10    geom_vline(xintercept = 2, linetype = "dashed") +
11    labs(x = "Estimated Variance") +
12    theme_bw()
13 }
14
15 p1 = make.plot(pois.data, "Poisson( = 2)")
16 p2 = make.plot(binom.data, "Binomial(n=50, p=(5-√21)/10)")
17 p3 = make.plot(norm.data, "Normal(0, √2)")
18 p4 = make.plot(exp.data, "Exponential( = 1/√2)")
19
20 # Combine plots
21 (p1 | p2) / (p3 | p4)

```

Figure 1: Comparison of Variance Estimators ($n = 20$)



Across all four distributions, $S_{(n-1)}^2$ is approximately unbiased (-0.04 to 0) while S_n^2 shows systematically more bias and no significant differences in variance, precision, or MSE.

2.2 Simulation over various sample sizes

Consider the effect of the sample size have on these results. Provide a graphical and numerical summary of what happens as n increases – say $n = 20$, $n = 50$, $n = 100$, and $n = 500$. Does the result in your initial simulation hold across samples? Or, do things change as the sample size changes? Ensure to set your randomization seed.

Solution

```
1 set.seed(7272)
2 sample.sizes = c(20, 50, 100, 500) #store sample sizes in a vector
3
4 summarize.results.n = function(data, dist.name, n) {
5   true.var = 2
6
7   tibble(
8     Distribution = dist.name,
9     n = n,
10    Estimator = c("S_{(n-1)}^2", "S_n^2"),
11    Bias = c(mean(data$s_n1) - true.var,
12            mean(data$s_n) - true.var),
13    Variance = c(var(data$s_n1),
14               var(data$s_n)),
15    Precision = c(1 / var(data$s_n1),
16                1 / var(data$s_n)),
17    MSE = c(
18      (mean(data$s_n1) - true.var)^2 + var(data$s_n1),
19      (mean(data$s_n) - true.var)^2 + var(data$s_n)
20    )
21  )
22 }
```

```

1 all.results = list()
2
3 #loop simulation function over sample sizes
4 for (n in sample.sizes) {
5   pois.data = simulate.variance(gen.pois, n)
6   binom.data = simulate.variance(gen.binom, n)
7   norm.data = simulate.variance(gen.norm, n)
8   exp.data = simulate.variance(gen.exp, n)
9
10  all.results[[as.character(n)]] = bind_rows(
11    summarize.results.n(pois.data, "Poisson", n),
12    summarize.results.n(binom.data, "Binomial", n),
13    summarize.results.n(norm.data, "Normal", n),
14    summarize.results.n(exp.data, "Exponential", n)
15  )
16 }
17 results.all.n = bind_rows(all.results)
18
19 results.all.n |>
20   knitr::kable(digits = 2)

```

Table 2: Bias, Variance, Precision, MSE of Variance Estimators across Sample Sizes

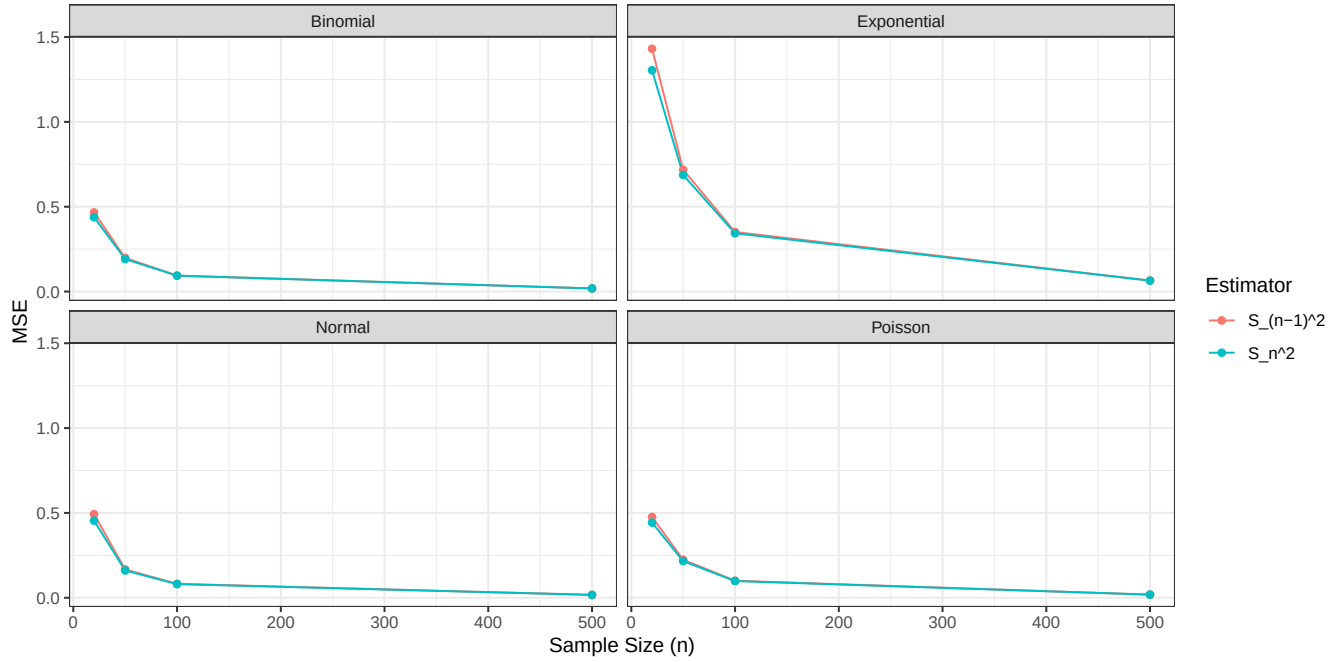
Distribution	n	Estimator	Bias	Variance	Precision	MSE
Poisson	20	S_{n-1}^2	-0.02	0.48	2.10	0.48
Poisson	20	S_n^2	-0.12	0.43	2.33	0.44
Binomial	20	S_{n-1}^2	-0.04	0.47	2.15	0.47
Binomial	20	S_n^2	-0.13	0.42	2.38	0.44
Normal	20	S_{n-1}^2	0.00	0.49	2.03	0.49
Normal	20	S_n^2	-0.10	0.44	2.25	0.45
Exponential	20	S_{n-1}^2	-0.02	1.43	0.70	1.43
Exponential	20	S_n^2	-0.12	1.29	0.77	1.30
Poisson	50	S_{n-1}^2	0.01	0.22	4.47	0.22
Poisson	50	S_n^2	-0.03	0.22	4.65	0.22
Binomial	50	S_{n-1}^2	0.00	0.20	5.05	0.20
Binomial	50	S_n^2	-0.04	0.19	5.26	0.19
Normal	50	S_{n-1}^2	0.01	0.17	6.00	0.17
Normal	50	S_n^2	-0.03	0.16	6.25	0.16
Exponential	50	S_{n-1}^2	0.05	0.72	1.40	0.72
Exponential	50	S_n^2	0.01	0.69	1.46	0.69
Poisson	100	S_{n-1}^2	0.01	0.10	9.96	0.10
Poisson	100	S_n^2	-0.01	0.10	10.17	0.10
Binomial	100	S_{n-1}^2	-0.03	0.09	10.73	0.09
Binomial	100	S_n^2	-0.05	0.09	10.94	0.09
Normal	100	S_{n-1}^2	0.02	0.08	12.20	0.08
Normal	100	S_n^2	0.00	0.08	12.45	0.08
Exponential	100	S_{n-1}^2	0.03	0.35	2.86	0.35
Exponential	100	S_n^2	0.01	0.34	2.91	0.34
Poisson	500	S_{n-1}^2	0.00	0.02	53.89	0.02
Poisson	500	S_n^2	0.00	0.02	54.11	0.02
Binomial	500	S_{n-1}^2	0.00	0.02	54.44	0.02
Binomial	500	S_n^2	0.00	0.02	54.66	0.02
Normal	500	S_{n-1}^2	0.00	0.02	58.39	0.02
Normal	500	S_n^2	0.00	0.02	58.63	0.02
Exponential	500	S_{n-1}^2	0.00	0.06	15.46	0.06
Exponential	500	S_n^2	0.00	0.06	15.52	0.06

```

1 ggplot(results.all.n, aes(x = n, y = MSE, color = Estimator)) +
2   geom_line() +
3   geom_point() +
4   facet_wrap(~ Distribution) +
5   labs(x = "Sample Size (n)") +
6   theme_bw()

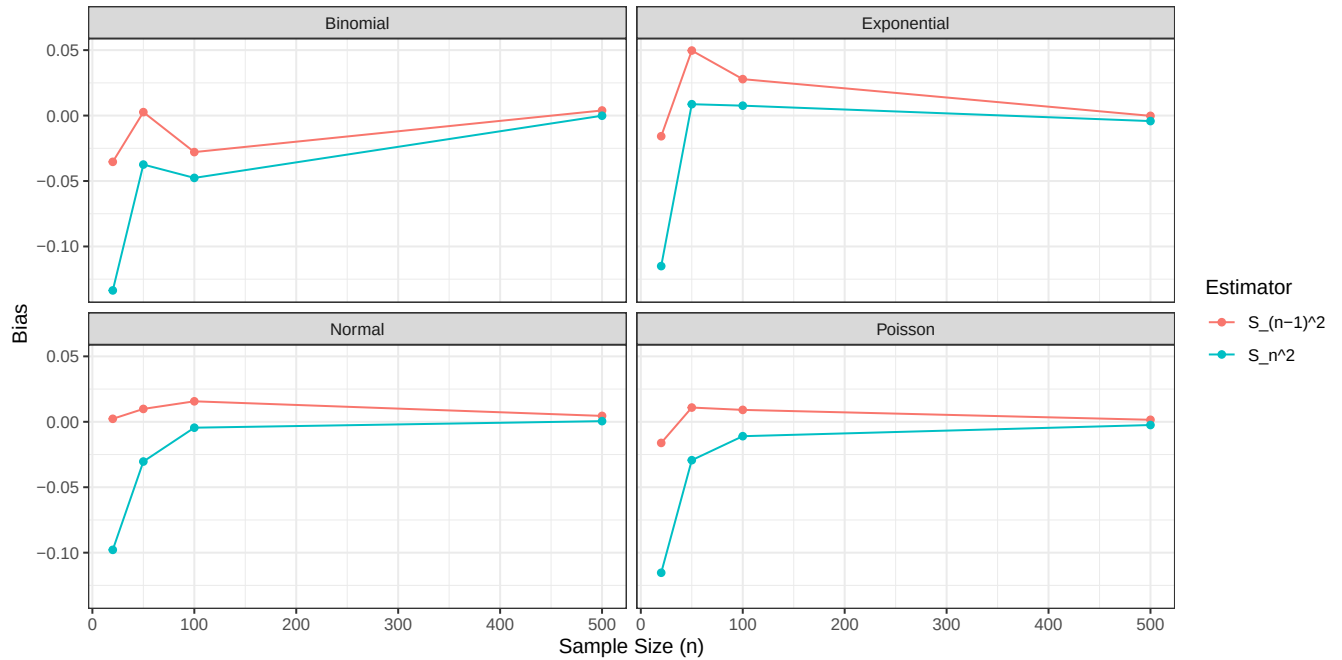
```

Figure 2: MSE of Variance Estimators vs Sample Size



```
1 ggplot(results.all.n, aes(x = n, y = Bias, color = Estimator)) +
2   geom_line() +
3   geom_point() +
4   facet_wrap(~ Distribution) + #facet by distribution
5   labs(x = "Sample Size (n)") +
6   theme_bw()
```

Figure 3: Bias vs Sample Size



As the sample size increases, bias for S_{n^2} decreases, while $S_{(n-1)^2}$ remains unbiased. It is especially noticeable that bias is approximately 0 for all $n=500$ samples. Variance and MSE also decreased for both as sample size increased, and precision increased. As sample size increases, it is less apparent which estimator is better. However,

across sample sizes S_n^2 systematically underestimates the true variance, whereas $S_{(n-1)}^2$ does not.

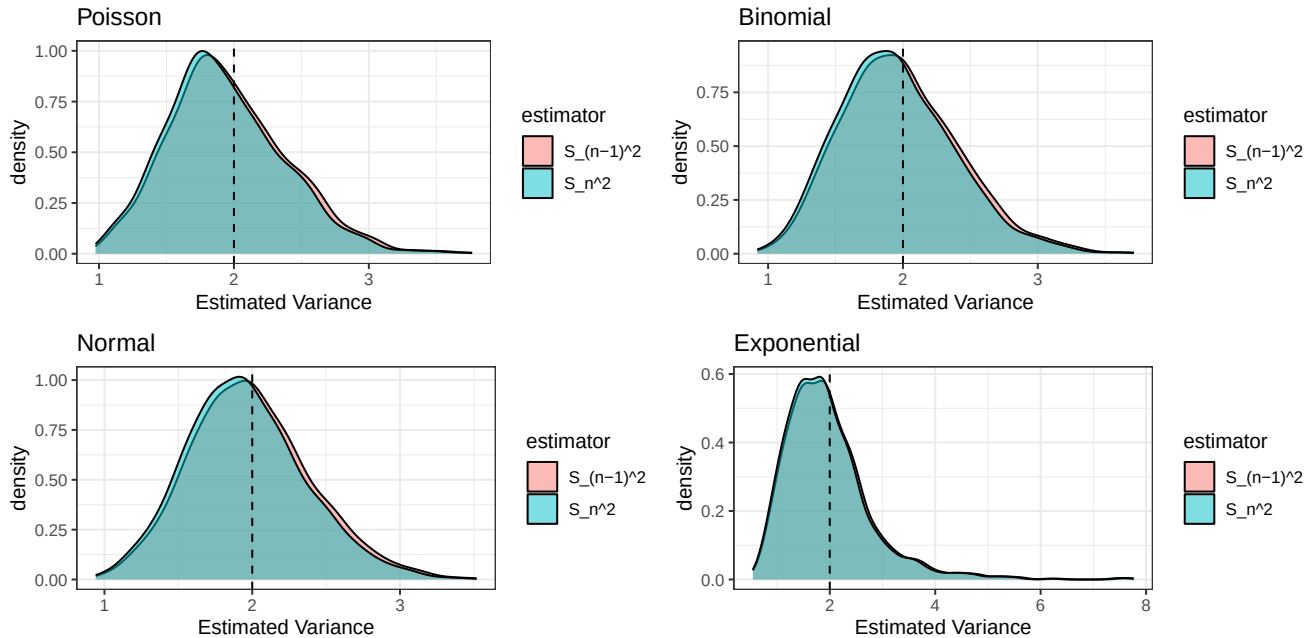
2.3 Simulation

You have written code to draw a random sample of various sizes from each distribution, and to calculate and store the calculated estimates. Rework your code to produce a density plot of the s_{n-1}^2 and s_n^2 . Do you have a guess about their distribution? Ensure to set your randomization seed.

Solution

```
1 set.seed(7272)
2
3 n = 50 #random n
4
5 pois.data = simulate.variance(gen.pois, n)
6 binom.data = simulate.variance(gen.binom, n)
7 norm.data = simulate.variance(gen.norm, n)
8 exp.data = simulate.variance(gen.exp, n)
9
10 #Plot function
11 make.density.plot = function(sim.data, title) {
12   df.plot = tibble(
13     value = c(sim.data$s_n1, sim.data$s_n),
14     estimator = rep(c("S_(n-1)^2", "S_n^2"), each = nrow(sim.data))
15   )
16
17   #Plot
18   ggplot(df.plot, aes(x = value, fill = estimator)) +
19     geom_density(alpha = 0.5) +
20     geom_vline(xintercept = 2, linetype = "dashed") +
21     labs(title = title, x = "Estimated Variance") +
22     theme_bw()
23 }
24
25 library(patchwork)
26
27 p1 = make.density.plot(pois.data, "Poisson")
28 p2 = make.density.plot(binom.data, "Binomial")
29 p3 = make.density.plot(norm.data, "Normal")
30 p4 = make.density.plot(exp.data, "Exponential")
31
32 (p1 | p2) / (p3 | p4) #Combine plots
```

Figure 4: Sampling Distributions of Variance Estimators



All four distributions appear to be slightly right skewed (even the normal distribution). This indicates that the skewness is a property of the variance estimators themselves rather than the original data distribution. While S_n^2 is slightly shifted to the left due to its negative bias, both estimators show similar overall shapes. As the sample size increases, these distributions would be expected to become more symmetric and concentrated around the true variance (2).

2.4 Bootstrap

Now, take *one* sample from each distribution. Instead of drawing a random sample at each iteration, sample *with replacement* from the one sample. Produce a density plot of the s_{n-1}^2 and s_n^2 . Do we get roughly the same information as we do in the full simulation? Try this over several seeds – what do you notice? Ensure to set your randomization seed.

Solution

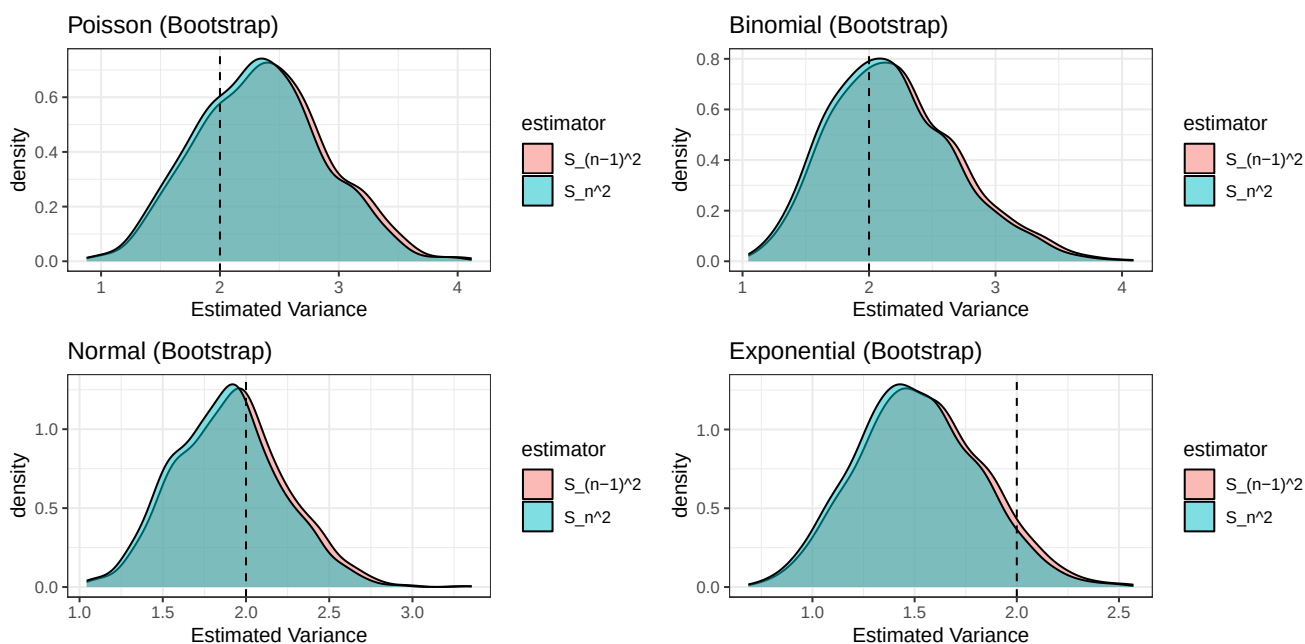
```
1 #Create bootstrap function
2 bootstrap.variance = function(x, n.boot = 1000) {
3   n = length(x)
4
5   s_n1 = numeric(n.boot)
6   s_n = numeric(n.boot)
7
8   for (i in 1:n.boot) { #loop sample
9     x.sample = sample(x, size = n, replace = TRUE) # bootstrap sample (replacement)
10
11     s_n1[i] = var(x.sample)
12     s_n[i] = mean((x.sample - mean(x.sample))^2)
13   }
14
15   tibble(s_n1 = s_n1, s_n = s_n)
16 }
1
2 #Take one random sample from each distribution
3 set.seed(7272)
```

```

3
4 n = 50
5
6 x.pois = rpois(n, 2)
7 x.binom = rbinom(n, 50, (5 - sqrt(21))/10)
8 x.norm = rnorm(n, 0, sqrt(2))
9 x.exp = rexp(n, rate = 1/sqrt(2))
10
11 pois.boot = bootstrap.variance(x.pois)
12 binom.boot = bootstrap.variance(x.binom)
13 norm.boot = bootstrap.variance(x.norm)
14 exp.boot = bootstrap.variance(x.exp)
15
16 p1 = make.density.plot(pois.boot, "Poisson (Bootstrap)")
17 p2 = make.density.plot(binom.boot, "Binomial (Bootstrap)")
18 p3 = make.density.plot(norm.boot, "Normal (Bootstrap)")
19 p4 = make.density.plot(exp.boot, "Exponential (Bootstrap)")
20
21 (p1 | p2) / (p3 | p4) #combine plots

```

Figure 5: Sampling Distributions of Variance Estimators Using Bootstrapping



The bootstrap distributions of $S_{(n-1)}^2$ and S_n^2 have similar shapes to those observed in the full simulation, including right-skewness and decreasing spread. This supports that bootstrapping is a useful method for approximating the sampling distribution using only one sample. However, the bootstrap distributions are centered around the sample variance rather than the population variance, so results can vary depending on the initial sample. When using different seeds, the overall shape remains similar, but the location and spread may change, reflecting variability in the original samples.

3 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) summary of the simulation and the results using your work in Lab 11. Your summary should be professional, clear, and all claims should be corroborated with statistical support.

Solution This lab investigated the properties of two variance estimators, $S_{(n-1)}^2$ and S_n^2 , through simulation and bootstrapping across four distributions. Using the Poisson, Binomial, Normal, and Exponential models, which

were all parameterized to have true variance 2, I repeatedly generated samples and computed both estimators. The results consistently showed that $S_{(n-1)}^2$ is approximately unbiased, with mean estimates very close to the true value of 2, while S_n^2 systematically underestimates the true variance. This bias was evident in the simulation summaries and density plots, where the distribution of S_n^2 was shifted slightly left of 2.

As sample size increased from $n=20$ to $n=500$, the variance and MSE of both estimators decreased, indicating improved precision. While the bias of S_n^2 diminished with larger n , it remained consistently negative. Consequently, the difference in performance between the two estimators was negligible for larger samples, though $S_{(n-1)}^2$ was unbiased for all sample sizes and did not exhibit persistent negative bias like S_n^2 did.

Density plots revealed that both estimators have right-skewed sampling distributions for small n , becoming more symmetric as n increases. Bootstrapping from a single sample produced similar distributional shapes, demonstrating that resampling can approximate the sampling distribution. However, bootstrap results were centered around the sample variance and varied across seeds, reflecting differences in the initial samples and potential inaccuracy with respect to the overall data. Overall, the simulations confirm the stated preference of using $S_{(n-1)}^2$, especially in smaller samples.